

DSA and SQL Interview Questions (0-3 Years) 5-25 LPA

DSA and SQL Questions

1. Write a program to check if a string of parentheses is valid.

A string is valid if:

- Open brackets are closed by the same type of brackets.
- Open brackets are closed in the correct order.

Example:

```
def isValid(s: str) -> bool:
```

```
    stack = []
```

```
    mapping = {'(':')', '[':']', '{':'}'
```

```
    for char in s:
```

```
        if char in mapping: # if closing bracket
```

```
            top = stack.pop() if stack else '#'
```

```
            if mapping[char] != top:
```

```
                return False
```

```
        else:
```

```
            stack.append(char) # if opening bracket
```

```
    return not stack
```

```
# Example test
```

```
print(isValid("()[]{}")) # True
```

```
print(isValid("()")) # False
```

Explanation:

- Use a **stack** to store opening brackets.
- When a closing bracket comes, check if it matches the top of the stack.
- If mismatched or stack is empty → invalid.
- At the end, if stack is empty → string is valid.
- This approach works for nested cases like "([{}])".

2. Write a program to reverse a linked list (Iterative & Recursive).

Example (Iterative Solution)::

```
def reverseList_iterative(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

Example (Recursive Solution):

```
def reverseList_recursive(head):
    if not head or not head.next:
        return head
    new_head = reverseList_recursive(head.next)
    head.next.next = head
    head.next = None
    return new_head;
```

- Output: All employees, with department info where available.

Explanation:

Iterative: Maintain pointers (prev, curr, next). Update links step by step until list is reversed.

Recursive: Reach the end, then reverse links while unwinding recursion.

Tip: Iterative uses **O(1) space**; Recursive uses **O(n) space** due to recursion stack. Always check for empty list or single node.

3. Write a program to search in a rotated sorted array using Binary Search.

```
def search(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            return mid

        # Left part is sorted
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        # Right part is sorted
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1;
```

Explanation:

- Normal binary search doesn't work directly because array is rotated.
- At each step, check which half (left or right) is **sorted**.
- Decide whether the target lies in the sorted half or the unsorted half.
- Narrow down the search until the element is found or range becomes invalid.
- This approach works in **$O(\log n)$** time.

4. Write a program to find the minimum window substring that contains all characters of another string (Sliding Window).

From collections import Counter

```
def minWindow(s: str, t: str) -> str:
    if not s or not t:
        return ""

    t_count = Counter(t)
    required = len(t_count)

    left, right = 0, 0
    formed = 0
    window_count = {}

    ans = float("inf"), None, None

    while right < len(s):
        char = s[right]
        window_count[char] = window_count.get(char, 0) + 1

        if char in t_count and window_count[char] == t_count[char]:
            formed += 1

        # Contract the window
        while left <= right and formed == required:
            if right - left + 1 < ans[0]:
                ans = (right - left + 1, left, right)

            # remove from window
            window_count[s[left]] -= 1
            if s[left] in t_count and window_count[s[left]] < t_count[s[left]]:
                formed -= 1
            left += 1

        right += 1

    return "" if ans[0] == float("inf") else s[ans[1]:ans[2]+1]

# Example test
print(minWindow("ADOBECODEBANC", "ABC")) # "BANC";
```

Explanation:

- Use two pointers (left, right) to form a sliding window.
- Expand the window by moving right until all characters of t are included.
- Then contract the window by moving left to minimize its size while still containing all required characters.
- Keep track of the smallest valid window.

5. Write a program to clone (deep copy) a graph using DFS

Class Node:

```
def __init__(self, val=0, neighbors=None):
    self.val = val
    self.neighbors = neighbors if neighbors else
```

```
[]
```

```
def cloneGraph(node: 'Node') -> 'Node':
```

```
    old_to_new = {}
```

```
    def dfs(n):
```

```
        if n in old_to_new:
```

```
            return old_to_new[n]
```

```
        copy = Node(n.val)
```

```
        old_to_new[n] = copy
```

```
        for nei in n.neighbors:
```

```
            copy.neighbors.append(dfs(nei))
```

```
    return copy
```

```
    return dfs(node) if node else None;
```

Explanation:

Use a hash map to map original nodes → cloned nodes.

DFS: Recursively traverse neighbors and clone them.

BFS: Use a queue to traverse level by level and clone nodes/edges.

Ensures no cycles cause infinite loops since already-visited nodes are tracked.

Tip:

- Works for connected and disconnected graphs.
- Always check if input node is None before cloning.
- BFS is iterative (avoids recursion depth issues), while DFS is more concise.

6. Write a program to find the median of two sorted arrays using Binary Search (Divide & Conquer)..

```
def findMedianSortedArrays(nums1, nums2):
    # Ensure nums1 is the smaller array
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)
    left, right = 0, m

    while left <= right:
        partitionX = (left + right) // 2
        partitionY = (m + n + 1) // 2 - partitionX

        maxLeftX = float('-inf') if partitionX == 0 else nums1[partitionX - 1]
        minRightX = float('inf') if partitionX == m else nums1[partitionX]

        maxLeftY = float('-inf') if partitionY == 0 else nums2[partitionY - 1]
        minRightY = float('inf') if partitionY == n else nums2[partitionY]

        # Correct partition
        if maxLeftX <= minRightY and maxLeftY <= minRightX:
            if (m + n) % 2 == 0:
                return (max(maxLeftX, maxLeftY) + min(minRightX, minRightY)) / 2
            else:
                return max(maxLeftX, maxLeftY)
        elif maxLeftX > minRightY:
            right = partitionX - 1
        else:
            left = partitionX + 1
```

Explanation:

- Use binary search on the smaller array to partition both arrays.
- Ensure elements on the left partition are $\leq$ elements on the right partition.
- If total length is odd $\rightarrow$ median is $\max(\text{left partition})$.
- If even $\rightarrow$ median is average of $\max(\text{left partition})$ and $\min(\text{right partition})$.
- This approach runs in **$O(\log(\min(m, n)))$** , much faster than merging arrays ($O(m+n)$).
- Always binary search on the **smaller array** to minimize complexity.

7. Write a program to implement a Trie (Prefix Tree).

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    # Insert word into Trie
    def insert(self, word: str) -> None:
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end = True

    # Search full word
    def search(self, word: str) -> bool:
        node = self.root
        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end

    # Check prefix exists
    def startsWith(self, prefix: str) -> bool:
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
            node = node.children[char]
        return True
```

Alternate using RANK() (if ties matter):

TrieNode: Stores children (dict) and is_end to mark word completion.

insert: Traverse characters; create nodes if they don't exist.

search: Check if full word exists and is_end is True.

startsWith: Only checks if prefix path exists in the Trie.

Trie is efficient for prefix-based problems (autocomplete, dictionary lookups).

Insert/search take **O(L)** time where L = length of word/prefix.

8. Explain the difference between UNION and UNION ALL.

Feature	UNION	UNION ALL
Duplicates	Removes duplicates	Keeps all rows, including duplicates
Performance	Slower (because of sorting)	Faster (no de-duplication)
Use case	When you want distinct rows	When duplicates are meaningful

Example:

```
SELECT city FROM customers
```

```
UNION
```

```
SELECT city FROM vendors;
```

→ Returns a unique list of cities.

```
SELECT city FROM customers
```

```
UNION ALL
```

```
SELECT city FROM vendors;
```

→ Returns **all cities**, including duplicates.

9. Write a program to find the Lowest Common Ancestor (LCA) of two nodes in a Binary Tree using DFS.

```
class TreeNode:
```

```
    def __init__(self, val=0, left=None, right=None):
```

```
        self.val = val
```

```
        self.left = left
```

```
        self.right = right
```

```
def lowestCommonAncestor(root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
```

```
    if not root or root == p or root == q:
```

```
        return root
```

```
    left = lowestCommonAncestor(root.left, p, q)
```

```
    right = lowestCommonAncestor(root.right, p, q)
```

```
    if left and right:
```

```
        return root
```

```
    return left if left else right;
```

Explanation:

- Traverse tree recursively.
- If current node is None or matches p or q, return it.
- Recursively search left and right subtrees.
- If both sides return non-null → current node is LCA.
- Else return whichever side is non-null.
- Works inside SELECT, WHERE, ORDER BY, etc.

10. Write a program to solve the Word Break Problem using Dynamic Programming.

```
def wordBreak(s: str, wordDict: list) -> bool:
    word_set = set(wordDict)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True # empty string is always valid

    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break

    return dp[n]

# Example test
print(wordBreak("leetcode", ["leet", "code"])) # True
print(wordBreak("applepenapple", ["apple", "pen"])) # True
print(wordBreak("catsandog", ["cats","dog","sand","and","cat"])) # False,
```

Explanation:

- Use DP array ($dp[i]$) = True if substring $s[0:i]$ can be segmented.
- For each index i , check all partitions $s[j:i]$.
- If $dp[j]$ is True and $s[j:i]$ is in dictionary $\rightarrow$ mark $dp[i] = \text{True}$.
- Final answer = $dp[n]$.
- Time complexity: $O(n^2)$, where n = length of string.
- For returning all possible sentences (Word Break II), use backtracking with memoization.

SQL

11. Write a query to retrieve the first order per customer.

```
SELECT customer_id, order_id, order_date
FROM (
    SELECT customer_id, order_id, order_date,
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date ASC) AS rn
    FROM orders
) t
WHERE rn = 1;;
```

Example (Using MIN with GROUP BY):

```
SELECT customer_id, MIN(order_date) AS first_order_date
FROM orders
GROUP BY customer_id;
```

Benefits:

- ROW_NUMBER() assigns a rank to each order per customer, ordered by order_date. Filtering rn = 1 gives the first order.
- Alternatively, MIN(order_date) with GROUP BY gives the earliest order date, but without details like order_id.
- Use JOIN if you need full order details with the MIN approach.
- Use ROW_NUMBER() if you need **order_id + other details** of the first order.
- Use MIN if you only need the date.
- RANK() or DENSE_RANK() can handle ties differently.

12. Write a query to retrieve the highest salary per department.

```
SELECT department_id, employee_id, salary
FROM (
    SELECT department_id, employee_id, salary,
           ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rn
    FROM employees
) t
WHERE rn = 1;
```

Example (Using MAX with GROUP BY):

```
SELECT department_id, MAX(salary) AS highest_salary
FROM employees
GROUP BY department_id;
```

Explanation:

- With ROW_NUMBER(), we partition data by department_id and order salaries in descending order. The first row (rn = 1) is the highest-paid employee per department.
- With MAX, we directly get the maximum salary per department but not the employee details.

Tip:

- Use ROW_NUMBER() if you need employee details along with the highest salary.
- Use MAX if you only need the **salary amount**.
- For ties (multiple employees with the same highest salary), use RANK() instead of ROW_NUMBER().

13. Write a query to find employees who don't have a manager.

Example (Using NULL check):

```
SELECT employee_id, employee_name
FROM employees
WHERE manager_id IS NULL;
```

Example (Using Self-Join):

```
SELECT e.employee_id, e.employee_name
FROM employees e
LEFT JOIN employees m
    ON e.manager_id = m.employee_id
WHERE m.employee_id IS NULL;
```

Explanation:

- In many schemas, top-level employees (like CEOs) have manager_id = NULL. A simple NULL check retrieves them.
- A **self-join** matches employees with their managers. If no matching manager exists, the employee has no manager.

Tip:

- Use NULL check if schema guarantees manager_id = NULL for no manager.
- Use **self-join** if you want to validate integrity (e.g., detect invalid manager_id values too).

14. Write a query to calculate the running total of salaries.

Example (Using SUM() OVER):

```
SELECT employee_id, salary,  
       SUM(salary) OVER (ORDER BY employee_id) AS running_total  
FROM employees;
```

Example (Department-wise running total):

```
SELECT department_id, employee_id, salary,  
       SUM(salary) OVER (  
         PARTITION BY department_id  
         ORDER BY employee_id  
       ) AS dept_running_total  
FROM employees;
```

Explanation:

- SUM() OVER (ORDER BY ...) computes a **cumulative sum** across ordered rows.
- Without PARTITION BY, it gives a global running total.
- With PARTITION BY department_id, it resets the running total for each department.
- **Tip:**
- You can change ORDER BY to hire_date or any other column for time-based running totals.
- For moving averages or sliding totals, use **ROWS BETWEEN** clause with window functions.

15. Write a query to find all customers who have not made any purchases in the last 6 months.

Assume:

- customers(customer_id, name)
- transactions(customer_id, transaction_date)

```
SELECT c.customer_id, c.name  
FROM customers c  
LEFT JOIN transactions t  
  ON c.customer_id = t.customer_id  
  AND t.transaction_date >= CURRENT_DATE - INTERVAL '6 months'  
WHERE t.customer_id IS NULL;
```

Explanation:

- LEFT JOIN includes all customers.

- WHERE t.customer_id IS NULL ensures the customer had **no purchase in the last 6 months**.
- Too many indexes can slow down INSERT/UPDATE.
- Avoid indexing columns with **low cardinality** (e.g., gender).
- Use **composite indexes** when querying multiple columns together.

PDF Owner Bhavesh Arora : [Bhavesh Arora | LinkedIn](#)

